THE LEGACY ASSESSMENT FRAMEWORK

A vendor-neutral instrument for assessing whether a system can still be changed - and for turning legacy from a surprise into a managed trajectory.

Version 1.0
May 2026
James Patrick CITP MBCS
Status: complete framework; known open areas identified in Section 9.

Contents

1. Purpose and framing

What this framework is for

Legacy is not old technology but technology that has become a persistent, unacceptable burden or risk to the public services that depend on it - it must be remediated because that burden compounds silently into cost, fragility and exposure, and it needs a shared framework because departments can only act rationally on it if they can see it, judge it and prioritise it the same way.

This framework exists to make that judgement consistent and defensible: it assesses a system or component across the dimensions that actually determine whether it can still be changed - its **optionality** - scores each on an anchored scale so two assessors reach the same answer, and tracks the result over time so decline is visible while there is still room to act, turning legacy from a surprise into a managed trajectory and from a verdict into a map.

Why “preventing legacy” is an incomplete frame

Legacy is not an event that good foresight could have averted. A system becomes legacy through the entirely normal accumulation of exogenous change: the surrounding ecosystem moves on, original engineers leave, business context shifts, security assumptions age out, dependencies go unsupported, underlying technologies change at unexpected pace. You could procure the right thing today, with impeccable judgement and best in class hyperscale components, and still hold legacy a decade later. Framing legacy as wholly preventable implies that its existence is a failure, which creates an incentive to conceal it rather than surface it - the opposite of what is needed.

The honest discipline is not prevention but evolvability and funded sustainment: accepting that everything built is a future legacy system, and that the meaningful question is how gracefully it can be changed, replaced, or retired. This framework measures exactly that.

Optionality: the property being measured

Optionality is the number of independent moves you can still make on a system without a coordinated, big-bang rebuild. A system with optionality can be moved, can have its parts developed and replaced separately, can have its integrations re-pointed, can be modernised, can be understood by new people, and can be changed with confidence. As optionality is lost - progressively, on different axes at different rates - the system trends toward legacy.

The framework's output is therefore not a binary verdict but a **profile**: it tells you *which* optionality has been lost, and therefore which intervention buys it back. A low score is not a failure to be concealed. It is a map.

The two directions of use

The same framework does two jobs depending on where in the lifecycle it is applied. Applied forward - at design and build - it is a design conformance standard: the tests become requirements the system is committed to meet, and the go-live score becomes a registered baseline. Applied backward - to an existing system - it is a condition assessment. Same

questions, opposite direction of causation. A new system that has never been assessed is not a special case; it is simply a system whose baseline has not yet been registered.

2. Structure of the framework

The framework is built in layers. Each layer earns the next, and they are applied in order:

1. **Seven headings** - the dimensions of optionality.
2. **Five anchored tests per heading** - thirty-five diagnostic questions, each scored 1–5.
3. **Gating tests** - designated tests that can floor a heading regardless of the others.
4. **Evidence indicators** - what must be shown for a score to stand.
5. **RAG ratings** - at heading, component, and system level.
6. **The load-bearing roll-up** - how component scores scale to a system honestly.
7. **Trajectory and cadence** - what makes the framework continuous rather than a snapshot.

The seven headings are ordered deliberately: technical properties of the artefact first (1–4), then knowledge properties (5–6), then the confidence-to-act dimension (7) that cuts across both. That sequence is itself an argument - it walks from the artefact, to the people, to whether you can actually move.

The seven headings at a glance

Heading	The question it asks
1. Portability	Can it be moved?
2. Modularity	Can one part change without forcing the rest to change?
3. Integration	Can its connections be changed plug-and-play?
4. Modernisation	Can the language and code be brought forward?
5. Embedded IP	Is critical bespoke logic buried and unextractable?
6. Knowledge Transferability	Can anyone pick it up and work with it?
7. Observability & Testability	Can you change it with confidence rather than on faith?

3. Calibration rules

These rules govern all scoring. They sit ahead of the tests because they are what make the framework defensible in a hostile room - a framework that cannot survive a programme team with a budget and a reputation arguing a red is not a red is not doing its job.

Demonstrated reality, not intent

The score reflects what is demonstrably true now, not what is planned or aspired to. “We plan to document it” is a 1 for documentation. Evidence beats assertion: if the assessor cannot see it, it is not a 5.

Score as-is, not as-designed

Score the system as it currently is, not as it was designed to be or could be with effort. The cost of restoring optionality is the output of the assessment, not an input to the score.

When uncertain, score down

When genuinely between two anchors, score the lower. Optimism bias is the mechanism by which the decade-long surprise happens. A defensible amber is worth more than a flattering green.

The scale

Every test is scored 1–5 against a closed, yes-leaning question. **1** = emphatic no, the optionality is gone. **3** = partial, qualified, or conditional. **5** = emphatic yes, the optionality is fully intact. A 2 sits between 1 and 3; a 4 between 3 and 5. The odd-numbered anchors in Section 4 are the load-bearing definitions; assessors interpolate the even numbers.

4. The seven headings and their tests

Each heading carries five tests with anchored descriptors for scores 1, 3 and 5. Tests marked **[G]** are gating tests: a low score on a gating test can floor the heading regardless of the other four (see Section 5). Evidence indicators for every test are in Section 6.

4.1 Portability

Can it be moved?

Test	Score 1	Score 3	Score 5
1.1 Run on other infrastructure without redesign [G]	Bound to its current host; moving it means a redesign.	Movable to similar infrastructure with significant rework or constraints.	Runs on alternative infrastructure with configuration changes only.
1.2 Runtime dependencies current and supported	Depends on out-of-support or end-of-life components.	Mostly supported; some components near end-of-life or on extended support.	All runtime dependencies are current and within their support lifecycle.
1.3 Free of hard coupling to proprietary services or hardware	Core function depends on proprietary services with no equivalent elsewhere.	Uses proprietary services, but behind abstractions or with known alternatives.	Uses portable, substitutable, or open interfaces throughout.
1.4 Can be stood up from definition, not hand-built	Environment exists only as a hand-built artefact; recreating it is reverse-engineering.	Partially defined as code; some manual steps or tribal knowledge required.	Fully reproducible from version-controlled definition.
1.5 Data extractable and migratable without bespoke tooling	Data is locked in a proprietary or undocumented store; extraction needs custom development.	Extractable, but with effort, format conversion, or partial bespoke tooling.	Stored in documented, standard formats; extractable with standard tools.

4.2 Modularity

Can one part change without forcing the rest to change?

Test	Score 1	Score 3	Score 5
2.1 Clearly bounded responsibilities	Responsibilities are diffuse; no clear statement of what it does and does not own.	Broadly bounded, but with overlaps or grey areas with other components.	Clear, documented, single area of responsibility.
2.2 Can be developed, built and released independently [G]	Cannot be built or released except as part of a larger coordinated effort.	Independently buildable, but release is coupled to others in practice.	Developed, built and released on its own cadence.
2.3 Internal parts separable, not rippling unpredictably	Internally entangled; any change risks unpredictable effects elsewhere.	Some internal structure; ripple is bounded but not fully predictable.	Cleanly partitioned internally; change effects are localised and predictable.

Test	Score 1	Score 3	Score 5
2.4 Shared mutable state avoided or tightly controlled	Shares mutable state freely; other components reach into its internals or vice versa.	Some shared state, but mediated or documented.	No uncontrolled shared state; interaction is through defined boundaries only.
2.5 A part can be replaced behind the same boundary	Implementation and interface are inseparable; replacing the part means rebuilding the boundary.	Replaceable with effort; the boundary leaks implementation detail.	Parts sit behind stable boundaries and can be swapped freely.

4.3 Integration

Can its connections be changed plug-and-play?

Test	Score 1	Score 3	Score 5
3.1 Integrates through defined, documented interfaces	Integrations rely on undocumented assumptions about behaviour, format, or timing.	Interfaces exist and are partly documented; some assumptions remain implicit.	All integrations through defined, documented interfaces.
3.2 Interfaces versioned so each side can change independently	No versioning; any interface change is a breaking change.	Some versioning or backward-compatibility discipline, inconsistently applied.	Interfaces are versioned; producers and consumers can evolve independently.
3.3 An integration can be re-pointed without modifying the core [G]	Integration endpoints are wired into the core; changing one means changing the component.	Re-pointable with configuration change plus some code change.	Integrations are externally configured; swappable without touching the core.
3.4 Free of brittle point-to-point couplings	Depends on undocumented timing, ordering, or sequence to function.	Some fragile couplings, known and worked around.	No reliance on incidental timing or order; couplings are explicit and robust.
3.5 Integration contracts testable independently	Integrations can only be tested against live counterpart systems.	Partially testable in isolation; some paths need the real counterpart.	Contracts can be fully exercised with stubs, mocks, or simulators.

4.4 Modernisation

Can the language and code be brought forward?

Test	Score 1	Score 3	Score 5
4.1 Language/framework supported and hireable for [G]	Language/framework is obsolete; hiring or retaining skills is a known problem.	Supported but ageing; skills pool is shrinking or expensive.	Current, actively supported, and readily hireable for.
4.2 Codebase structured so change is safe, not feared	Change is feared; the team avoids touching it where possible.	Change is possible but slow and carries known risk.	Change is routine and made with confidence.

Test	Score 1	Score 3	Score 5
4.3 Toolchain can still pull current, patched components	Toolchain is frozen; pulling current or patched dependencies is impossible or breaks the build.	Toolchain works but is dated; updates are difficult.	Toolchain is current and pulls patched dependencies cleanly.
4.4 Free of dead, unexplained code no one dares remove	Significant dead or unexplained code that no one will remove.	Some dead or unclear code, identified but not cleared.	Codebase is clean; no significant dead or unexplained sections.
4.5 A section could be rewritten without reverse-engineering it	Rewriting any section first requires reverse-engineering what it does.	Rewritable from a mix of documentation and code reading.	Behaviour is specified well enough to rewrite directly.

4.5 Embedded IP

Is critical bespoke logic buried and unextractable?

Note. *High score = the IP is well-handled and extractable. Score "how well-handled is the IP", not "how much buried IP is there". Demonstration register: a document describing the IP is not evidence the IP is extractable.*

Test	Score 1	Score 3	Score 5
5.1 Bespoke logic documented to the standard of its complexity [G]	Critical bespoke logic is undocumented; it exists only as code.	Partially documented; documentation lags the code or covers only the simple parts.	Documented proportionate to its complexity and kept current.
5.2 Logic isolated and identifiable, not smeared through general code	Bespoke logic is smeared through the codebase; you cannot point to where it lives.	Mostly identifiable, with some logic embedded in general-purpose code.	Isolated, identifiable, and clearly delineated from general-purpose code.
5.3 The IP could be re-implemented from its specification	No specification exists; the code is the only statement of the IP.	A specification exists but is incomplete or partly out of date.	A complete, current specification exists independent of the code.
5.4 Free of undocumented edge-case handling encoding lost decisions	Contains undocumented edge-case handling whose rationale is lost.	Some edge cases undocumented, but rationale is recoverable from people or records.	Edge cases and their rationale are documented.
5.5 The IP can be tested against a known-correct specification	No way to prove a reimplementation is correct; no reference to test against.	Partial reference or test set; would prove some but not all behaviour.	A reference specification or test set exists sufficient to validate a reimplementation.

4.6 Knowledge Transferability

Can anyone pick it up and work with it?

Note. *Demonstration register: documentation describing the system is not evidence the knowledge is transferable. Transfer must have been demonstrated.*

Test	Score 1	Score 3	Score 5
6.1 Understanding spread across enough people to survive turnover [G]	Understanding sits with one person; their	Understood by a small group; thin but not a single point of failure.	Understanding is spread across enough people to absorb turnover.

Test	Score 1	Score 3	Score 5
	departure is a single point of failure.		
6.2 Onboarding possible from documentation, not only the incumbent	Onboarding is only possible by sitting with the incumbent.	Documentation gets someone partway; the rest is tribal.	A capable person can onboard from documentation alone.
6.3 Original design decisions and trade-offs recorded	Only the current state is visible; the “why” is lost.	Some rationale recorded; gaps in the decision history.	Design decisions and trade-offs are recorded and findable.
6.4 For cleared/constrained environments, enough eligible people hold the knowledge	Knowledge is held by too few eligible (cleared/qualified) people to sustain the system.	Eligible coverage is thin or depends on a small number of constrained individuals.	Enough eligible people hold the knowledge to sustain and change the system.
6.5 Operational knowledge captured as well as build knowledge	Run, recover and troubleshoot knowledge is uncaptured; it lives only with operators.	Operational knowledge is partly captured; recovery or edge scenarios are gaps.	Operational knowledge is captured to the standard of the build knowledge.

4.7 Observability & Testability

Can you change it with confidence rather than on faith?

Note. *Demonstration register. Evidence should be sourced from system telemetry wherever possible - this makes the heading the natural basis of the recurring light-check mode (see Cadence).*

Test	Score 1	Score 3	Score 5
7.1 Behaviour and health can be observed in operation [G]	Effectively a black box in operation; behaviour and health are not visible.	Partial visibility; gaps in what can be observed.	Behaviour and health are observable across the component.
7.2 Test harness sufficient to catch regressions	No meaningful test coverage; regressions are found in production or by users.	Partial coverage; catches some regressions, misses others.	Test coverage is sufficient to catch regressions before release.
7.3 Deploy and rollback are routine, not high-ceremony events	Deployment is a rare, high-ceremony, high-risk event; rollback is unreliable or absent.	Deployment is manageable but heavy; rollback is possible but not routine.	Deploy and rollback are routine, low-drama operations.
7.4 A change can be validated in a representative environment	No representative environment; changes are validated in production.	A test environment exists but diverges from production in ways that matter.	A representative environment exists where changes are validated before production.
7.5 Accreditation/assurance burden is proportionate enough that change happens [G]	Assurance burden is so high that change is avoided; the system ossifies regardless of its technical state.	Burden is significant and slows change, but change still happens.	Assurance is proportionate and built into the change process; it does not block evolution.

5. Gating tests and heading scores

Within a heading, a simple average lets strong answers mask fatal ones - five 5s and five 1s should not read as a comfortable 3 over a landmine. Each heading therefore carries one or more designated gating tests, marked [G] in Section 4. A low score on a gating test floors the heading regardless of the other tests.

The principle is consistent at every scale of the framework: the weakest load-bearing element governs. Gating tests govern the heading the way load-bearing components govern the system (Section 7). It is one rule applied at two levels, not two rules.

The non-gating tests within a heading are aggregated to give the heading its texture and its trajectory. The gating tests determine whether that aggregate is allowed to stand. Because every gating test has an anchored descriptor (Section 4) and an explicit evidence indicator with an absence clause (Section 6), a floored heading is always backed by a concrete, stated condition the assessor can point at - not merely a low number.

The twelve gating tests

Heading	Gating tests
1. Portability	1.1 Run on other infrastructure without redesign
2. Modularity	2.2 Can be developed, built and released independently
3. Integration	3.3 An integration can be re-pointed without modifying the core
4. Modernisation	4.1 Language/framework supported and hireable for
5. Embedded IP	5.1 Bespoke logic documented to the standard of its complexity
6. Knowledge Transferability	6.1 Understanding spread across enough people to survive turnover
7. Observability & Testability	7.1 Behaviour and health can be observed in operation 7.5 Accreditation/assurance burden is proportionate enough that change happens

Heading 7 carries two gating tests. The first because an unobservable component is unchangeable in practice; the second because in regulated and accredited environments the assurance burden is frequently the actual ossifying force even when every other axis scores well - a system can be modular, portable and well-understood and still be unable to change.

6. Evidence indicators

The anchored descriptor tells an assessor what a score means. The evidence indicator tells the assessor what must be shown to claim it. This layer is what turns “score down when uncertain” from a discouragement into a mechanism.

The governing principle

Evidence is something a second person could independently verify exists. An assertion by the system owner is not evidence - it is the claim the evidence is meant to support.

The indicator answers one question: what would I have to be shown to believe this score? If the honest answer is “nothing, I would take their word for it”, the test cannot be scored above its floor.

Two registers of evidence

- **Artefact evidence** - a document, pipeline, configuration repository, test report or manifest exists and can be inspected. Most of headings 1–4 are artefact-evidenced.
- **Demonstration evidence** - the claim is proven by the thing having been done, not by a document about it. “Deploy and rollback are routine” is evidenced by logs showing it happened repeatedly, not by a runbook. Headings 5, 6 and 7 are demonstration-register, and this is stated at those headings in Section 4 because they are precisely where an assessor's instinct will be to accept a document as proof.

Evidence not available

When an assessor genuinely cannot obtain the evidence - records lost, no access, owner unresponsive - the score is not “skip” and not an average. It is the floor, flagged as evidence-absent rather than assessed-low. This removes the incentive to make evidence hard to find, and it is diagnostically honest: a component whose evidence cannot be obtained is, on that axis, as unsafe to act on as one that scored low. The flag preserves the distinction for whoever reads the profile; the score does not let the gap hide.

The evidence indicator is written once, centrally. Assessors do not invent it. Their task is to record whether evidence was present, partial, or absent, and to cite what it was - for example, “scored 4 on 4.3; evidence: CI pipeline plus eighteen months of dependency-update commits”. That citation is what makes a score survive challenge: the dispute must then be about the evidence, not the number.

6.1 Portability - evidence indicators

Test	Evidence indicator
1.1 [G]	**Evidence:** The component running, or having run, on a second distinct environment; or an architecture record identifying host dependencies and showing they are abstracted. ***Absence of evidence:*** *No second environment can be named and no design record addresses portability - floor.*

Test	Evidence indicator
1.2	**Evidence:** A current dependency/version inventory cross-referable to vendor or community support lifecycle dates.
1.3	**Evidence:** An architecture record or dependency map showing external service calls and their substitutability, or evidence those interfaces are abstracted.
1.4	**Evidence:** Version-controlled infrastructure/config definitions, ideally with a record of the environment having been rebuilt from them.
1.5	**Evidence:** Documented data formats and schema, plus a record of a successful standard-tool extract/migration or a defensible assessment that one is possible.

6.2 Modularity - evidence indicators

Test	Evidence indicator
2.1	**Evidence:** A design or service record stating what the component does and does not own, recognisable in the actual code structure.
2.2 [G]	**Evidence:** A build/release record showing it has been released on its own, separate from other components. ***Absence of evidence:*** *Every release on record is part of a coordinated multi-component effort - floor.*
2.3	**Evidence:** Internal structure visible in the codebase, plus change history showing past changes stayed localised.
2.4	**Evidence:** A data/interaction map showing what state is shared and how access is mediated; or demonstrable absence of components reaching into each other's internals.
2.5	**Evidence:** Defined internal interfaces, ideally with a record of a part having been swapped without rebuilding the boundary.

6.3 Integration - evidence indicators

Test	Evidence indicator
3.1	**Evidence:** Interface definitions or contracts that exist as inspectable artefacts, not descriptions of behaviour after the fact.
3.2	**Evidence:** A versioning scheme visible in the interface artefacts, plus history of a version change managed without breaking counterparts.
3.3 [G]	**Evidence:** Integration endpoints held in external configuration, ideally with a record of one having been re-pointed without a code change. ***Absence of evidence:*** *Endpoints are wired into the component and any change requires modifying it - floor.*
3.4	**Evidence:** Integration design showing explicit sequencing/handling; or demonstrable absence of incident history caused by timing or ordering.
3.5	**Evidence:** Stubs, mocks, simulators, or a contract-test suite that exists and is exercisable without the live counterpart.

6.4 Modernisation - evidence indicators

Test	Evidence indicator
4.1 [G]	**Evidence:** The language/framework version cross-referable to a current support lifecycle, plus a defensible view of the skills market. ***Absence of evidence:*** *Version is unidentified or demonstrably out of support and the skills position is unknown - floor.*
4.2	**Evidence:** Change lead-time and change-failure history; or a code-quality assessment. Team assertion that change is "fine" is not evidence - the change record is.
4.3	**Evidence:** A working build that demonstrably pulls current dependencies; or a recent successful dependency/patch update on record.
4.4	**Evidence:** A code analysis identifying dead/unreachable code; or a clean assessment. A claim of cleanliness with no analysis behind it is not evidence.
4.5	**Evidence:** Behavioural specifications or documentation sufficient to restate what a section does independently of reading the code.

6.5 Embedded IP - evidence indicators

Register note. *High score = the IP is well-handled and extractable. Score "how well-handled is the IP", not "how much buried IP is there". Demonstration register: a document describing the IP is not evidence the IP is extractable.*

Test	Evidence indicator
5.1 [G]	**Evidence:** Documentation that demonstrably matches current behaviour - ideally validated by someone using it to predict or explain an output correctly. ***Absence of evidence:*** *The logic exists only as code, or documentation cannot be shown to be current - floor.*
5.2	**Evidence:** An assessor or engineer can point to where the bespoke logic lives, distinct from general-purpose code.
5.3	**Evidence:** A specification exists as an artefact independent of the code, complete enough that re-implementation would not require reading the code.
5.4	**Evidence:** Edge cases and their rationale are recorded; or demonstrable confidence - via people or records - that the rationale behind existing edge-case handling is recoverable.
5.5	**Evidence:** A reference specification or test set exists that would let a re-implementation be validated as behaviourally correct.

6.6 Knowledge Transferability - evidence indicators

Register note. *Demonstration register: documentation describing the system is not evidence the knowledge is transferable. Transfer must have been demonstrated.*

Test	Evidence indicator
6.1 [G]	**Evidence:** More than one person has independently made a non-trivial change; or the component has demonstrably survived a relevant departure. ***Absence of evidence:*** *All evidence points to a single individual as the only person who has worked on it - floor.*

Test	Evidence indicator
6.2	**Evidence:** A documented instance of someone onboarding from the documentation; or documentation an assessor judges sufficient against that test. Existence of documentation is not the same as it being sufficient.
6.3	**Evidence:** Decision records, architecture rationale, or equivalent capturing the “why”, not only the current state.
6.4	**Evidence:** A demonstrable count of eligible (cleared/qualified) individuals who hold working knowledge sufficient to sustain and change the system. Score N/A only where no clearance or eligibility constraint genuinely applies - and that exemption must itself be justified, not assumed.
6.5	**Evidence:** Operational runbooks plus demonstration they work - e.g. a recovery actually performed from them, or troubleshooting carried out by someone other than the original operator.

6.7 Observability & Testability - evidence indicators

Register note. *Demonstration register. Evidence should be sourced from system telemetry wherever possible - this makes the heading the natural basis of the recurring light-check mode (see Cadence).*

Test	Evidence indicator
7.1 [G]	**Evidence:** Live monitoring, logging, or telemetry output that an assessor can actually view. ***Absence of evidence:*** *The component emits nothing inspectable and its operational state is inferred indirectly - floor.*
7.2	**Evidence:** A test suite plus coverage data and a history of it actually catching regressions before release.
7.3	**Evidence:** Deployment and rollback logs showing both happening repeatedly without incident. A runbook describing the process is not evidence it is routine.
7.4	**Evidence:** A test/staging environment that exists, plus evidence of how closely it matches production and of changes having been validated there.
7.5 [G]	**Evidence:** The actual elapsed time and cost of recent changes through the assurance process - the demonstrated fact that change still happens. ***Absence of evidence:*** *No change has completed the assurance process recently enough to judge, or the process cannot be timed - floor, flagged, because an un-timeable assurance process is itself the ossifying condition.*

7. Ratings and the system-level roll-up

The three ratings

Each heading, each component, and each system resolves to one of three ratings.

Rating	Meaning
GREEN	**Evolvable** Optionality is substantially intact. The system or component can still be changed, moved, and developed without a coordinated rebuild. No action required beyond continued sustainment.
AMBER	**In natural selection** Optionality has been lost on at least one axis. This is a decision-required state, not a holding pen: amber carries a mandatory review date and a recorded choice - invest to restore optionality, or consciously accept the trajectory and plan the exit. Amber without a review date is red in denial.
RED	**Remediation required** Optionality has been lost to the point where the system or component cannot be safely evolved. The rational economic move is exit or rebuild, not continued life support. Every red is specific: it names the axis on which optionality was lost, and therefore its own remediation.

Amber is the rating most open to abuse, because green and red are self-explanatory and hard to game. Amber is defined here not as a holding pen but as a **decision-required state with a clock on it**. Amber means optionality has been lost on at least one axis and a live, time-bound choice now exists. Amber that is stable and consciously accepted, with a review date, is a system being managed. Amber that is declining, or amber with no review date, is the failure mode.

Classifying structural role

Before any roll-up, each component is given a dependency classification - not a score, a category. This classification is itself a small assessment, and it must be recorded and open to challenge, because it is where a programme team will try to argue a red component is merely “contributing”. Tie it to evidence: if there is no alternative path, the component is load-bearing, however inconvenient that is.

Classification	Definition
Load-bearing	The system cannot perform its core function without this component, and there is no ready alternative path. Its failure or ossification is the system's failure or ossification.
Contributing	The component matters to the system's function or quality, but the system degrades rather than fails without it, or an alternative path exists.
Peripheral	The component supports the system but is replaceable, optional, or isolated enough that its state does not constrain the system's overall optionality.

The roll-up rule

The system-level rating is set by the worst rating among its load-bearing components. A system with any load-bearing component at red is a red system, because the system has in fact lost the ability to evolve without first dealing with that component. A flat average would report nine pristine components and one terminal load-bearing component as a comfortable amber - reassuring and wrong. Taking the worst of all components, regardless of role, would cry wolf at every system with one ageing peripheral part. The worst-load-bearing rule sits honestly between the two: it is a statement about reality, not a posture of pessimism.

Contributing and peripheral components do not set the system rating, but they are not discarded. The averaged heading profile across all components is always presented alongside the rating, as context: a system that is red on one load-bearing component but green everywhere else is a targeted remediation job; a system that is red on one and high-amber across everything else is a system in trouble. Same headline rating, very different reality - the profile is what distinguishes them.

Why there is no numeric weighting - yet

The roll-up is deliberately categorical, not a weighted formula. “Your system is red because a component with no alternative path is red” is a sentence no one can wriggle out of. “Your system is red because the weighted index came to 2.7” invites a fight about the 2.7. A numeric weighting model may later be added on top of this - as a tiebreaker for ranking multiple red systems against a finite remediation budget - but the spine stays categorical, because categorical is what survives a hostile review. This is recorded as a known open area in Section 9.

8. Trajectory and cadence

A snapshot rating tells you where a component is. It does not tell you whether to be worried - a new system scores green by definition, and a component can sit at stable amber for years quite legitimately. The decade-long surprise does not happen because nobody scored a system; it happens because nobody scored it twice and compared. The framework's continuous value lives in the delta.

The score is the state; the delta is the signal

Every assessment after the first produces two outputs: the current rating, and the delta against the previous assessment. The delta is what gets escalated. It is tracked at heading level, not only whole-component level, because the shape of a decline names its cause: Modernisability sliding tells a different story - language ageing, toolchain freezing - from Knowledge Transferability sliding - people leaving, documentation rotting - and the two need different interventions.

Trajectory state	What it means
Stable	Score unchanged or within noise. Action depends on the rating: stable green is healthy; stable amber is legitimate only if consciously accepted with a review date; stable red is an unaddressed remediation case.
Declining	Score dropping on one or more headings. This is the warning state regardless of current rating. A component sliding green-to-amber in eighteen months is a louder signal than one that has sat at amber for five years. This is the state the framework exists to catch.
Improving	Score rising. Recorded because it is the evidence base that remediation or evolution investment is working.
Volatile	Score moving up and down between assessments. Often the most diagnostic: usually means the component is being actively changed without stable architecture underneath, or that assessment quality itself is inconsistent.

Two modes of assessment

If the full thirty-five-test assessment is the only mode, it will not be run continuously - it will be run in a panic, which is the surprise cycle with extra steps. The framework therefore has two modes.

Mode	Description
Full assessment	All thirty-five tests. Run at lifecycle inflection points - design baseline, go-live, major architectural change, significant integration change - and on a periodic deep-review cycle: at least annually for load-bearing components, less often for stable low-criticality ones. This is the authoritative score and resets the trajectory baseline.
Light check	The twelve gating tests only, fed wherever possible from telemetry the system already emits (see the register note at Heading 7). Run frequently - quarterly, or continuously where telemetry allows. Its job is not to produce an authoritative rating but to detect movement on the tests most likely to floor a heading, and to trigger a full assessment when it sees movement. The light check is the smoke detector; the full assessment is the inspection.

The baseline obligation

Every new system receives a full assessment at design and at go-live, and the go-live score is its registered baseline. This is non-optional. It commits the system to a known starting optionality - no one can later claim it was “always like that” - and it guarantees that no system exists without a trajectory. Make the baseline mandatory at go-live and the decade-long surprise becomes structurally impossible: the worst case becomes a visible decline rather than an invisible one.

A caution on calibration

Trajectory only means something if scores are calibrated and assessor-consistent - a delta between two differently-biased assessors is noise, not signal, and a lying trajectory is worse than none because it is trusted. The anchored descriptors and evidence indicators do most of this work. Beyond them, keep the same assessor or assessment team across consecutive assessments of a component wherever possible.

9. Known open areas

A framework that is honest about its edges is more credible than one that pretends to be finished. Two areas are deliberately left open in version 1.0.

Numeric dependency weighting

The roll-up in Section 7 is categorical: the worst load-bearing component sets the system rating. This is the right spine because it is defensible and hard to game. It does not, however, finely rank multiple red systems against each other - which is what is needed to sequence a finite remediation budget. A numeric weighting model layered on top of the categorical rule, used strictly as a tiebreaker, is the natural next increment. It is left out of version 1.0 on purpose: introducing it too early would trade away the defensibility that makes the framework work.

Assessor competence

This framework defines what evidence looks like but not who is competent to judge it. An evidence indicator only works if the assessor can recognise whether what they have been shown actually meets the bar - “more than one person has made a non-trivial change” requires the assessor to judge what is non-trivial. An assessor-competence and moderation model is the natural layer after this one. Until it exists, consistency depends on the anchored descriptors, the evidence indicators, and continuity of assessors across assessments.

10. Applying the framework

The framework is a single instrument with several uses. The instrument does not change between uses; only who it is handed to does.

Assessment workflow

1. Classify each component's structural role (load-bearing, contributing, peripheral) and record the basis for the classification.
2. For each component, score all thirty-five tests 1–5 against the anchored descriptors, citing the evidence for each score. Apply the calibration rules: demonstrated reality, score as-is, score down when uncertain.
3. Resolve each heading: apply gating tests, then aggregate the non-gating tests for texture. A floored gating test floors the heading.
4. Resolve each component to a RAG rating from its heading ratings.
5. Roll up to the system: the worst rating among load-bearing components sets the system rating. Present the averaged all-component profile alongside it as context.
6. Record the result against the component's baseline and previous assessment to produce the trajectory. For any amber, record a review date and the decision taken.

Discovery and lightweight use

For a fast, indicative read - in discovery, early engagement, or a workshop - run the twelve gating tests only across a small number of representative systems, and label the output explicitly as indicative. This produces a real optionality picture and a clear view of where a fuller assessment would pay back, without the cost of a full thirty-five-test exercise. The full framework is what the indicative read points to when its rigour is questioned.

A note on neutrality

The framework measures optionality. It does not road toward any product, supplier, or remediation approach. A component can score green and need nothing; a red can have a remediation path that is not a migration at all. This neutrality is not a constraint on the framework - it is the source of its authority. A rating is only worth defending if the framework was capable of returning the answer no one wanted.

End of framework, version 1.0.